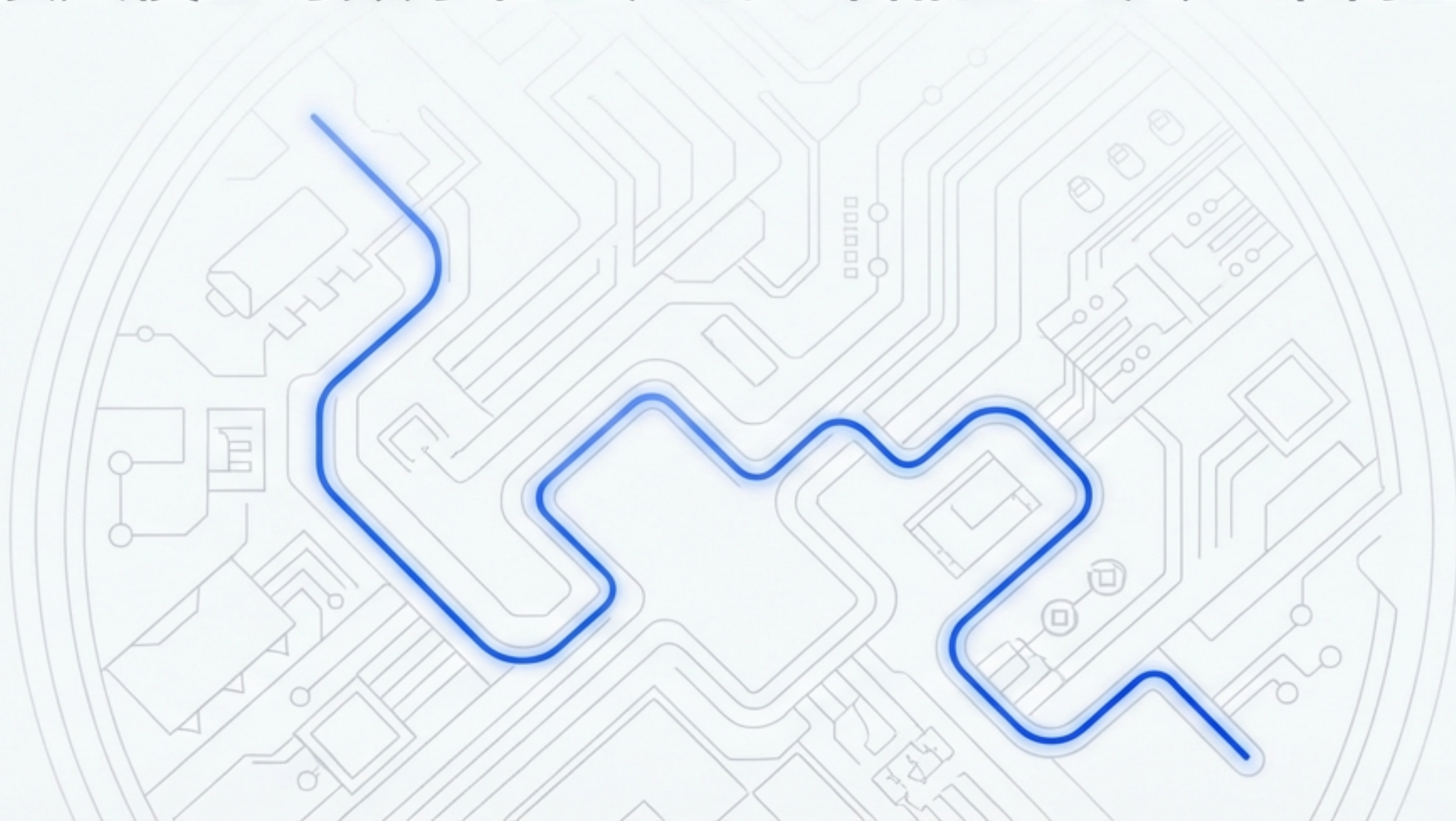


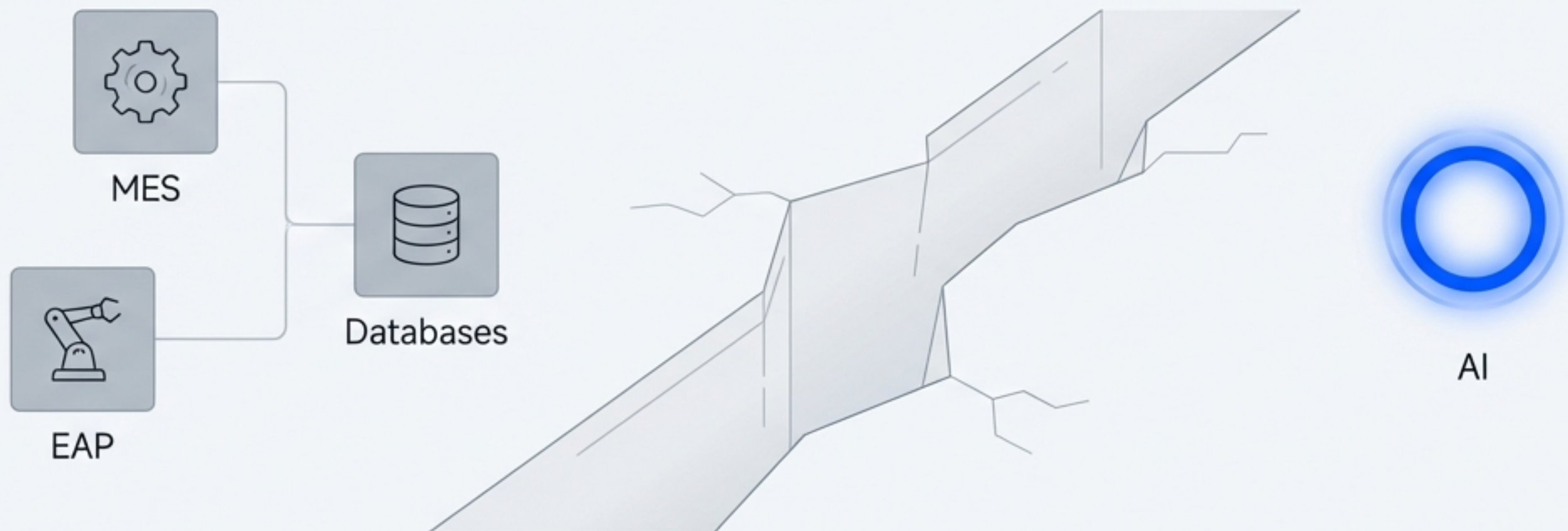
构建 AI 友好的未来工厂

连接大模型与数字化工厂的“智能适配层”架构蓝图



我们已实现高度数字化，但系统并非为 AI 而设计

我们拥有庞大的 MES、EAP 和海量数据，但现有系统是为“工程师”或“脚本”设计的，而非“AI”。这导致了一个巨大的鸿沟。



数据孤立

AI 无法理解分散在不同数据库和日志中的数据之间的关联。



接口复杂

数千个晦涩的 API 对 AI 来说是“黑盒”，调用风险极高。

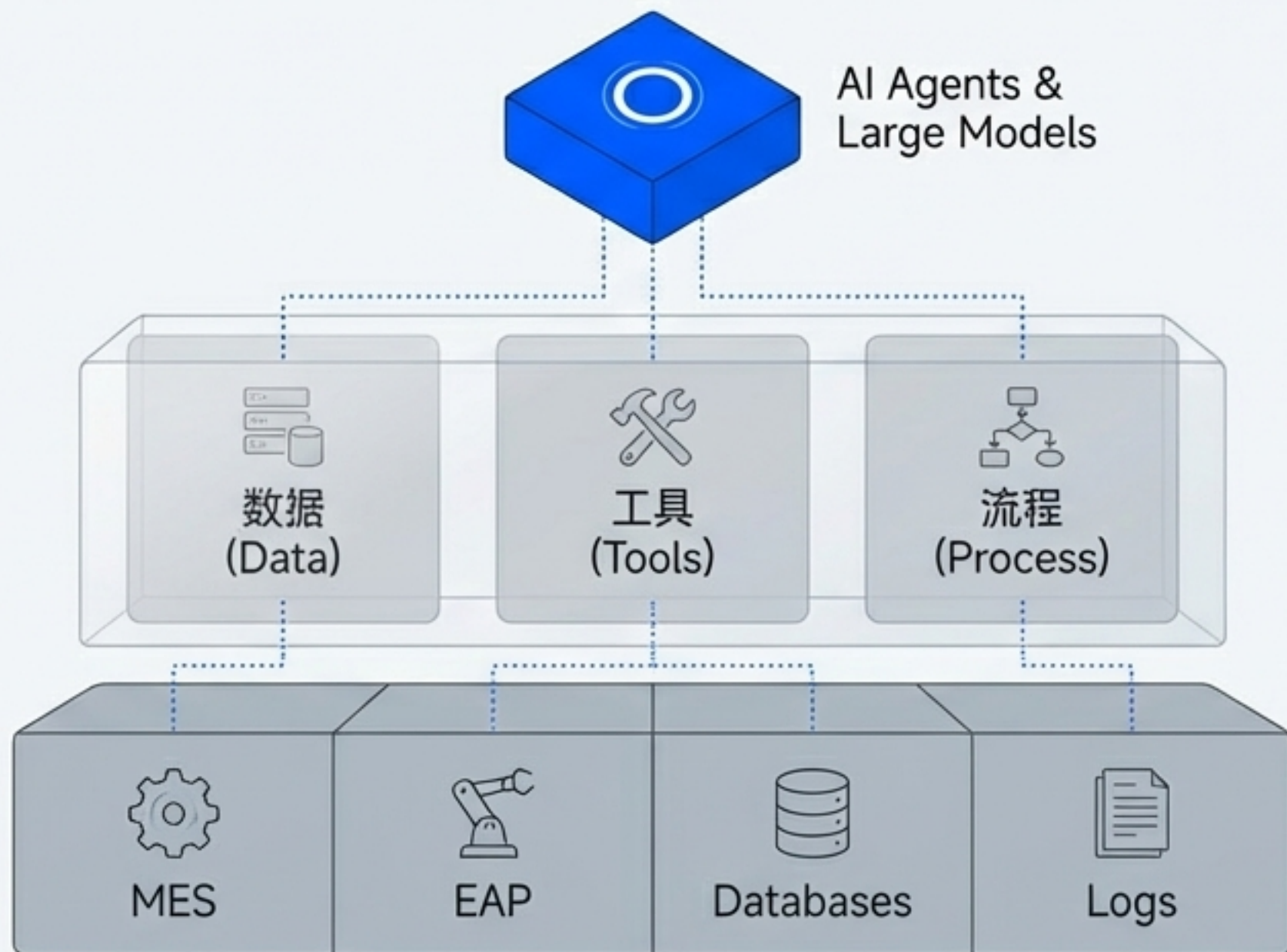


流程僵化

SOP 写在 PDF 中，供人死记硬背，无法被 AI 动态执行。

解决方案：构建一个零侵入式的“智能适配层”

我们无需推翻现有的数字化基建。我们要做的是在其之上，构建一个轻量级、标准化的智能适配层（Intelligence Layer），进行一场**上下文工程（Context Engineering）**的架构升级。



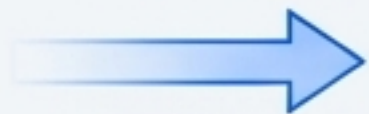
鸿沟：我们不缺数据，缺的是“模型可理解的上下文”

传统的做法（如将 Log 直接扔进向量数据库）效果不佳，因为工业数据高度关联，简单的文本切片会破坏其内在逻辑。AI 得到的是碎片，而非洞察。

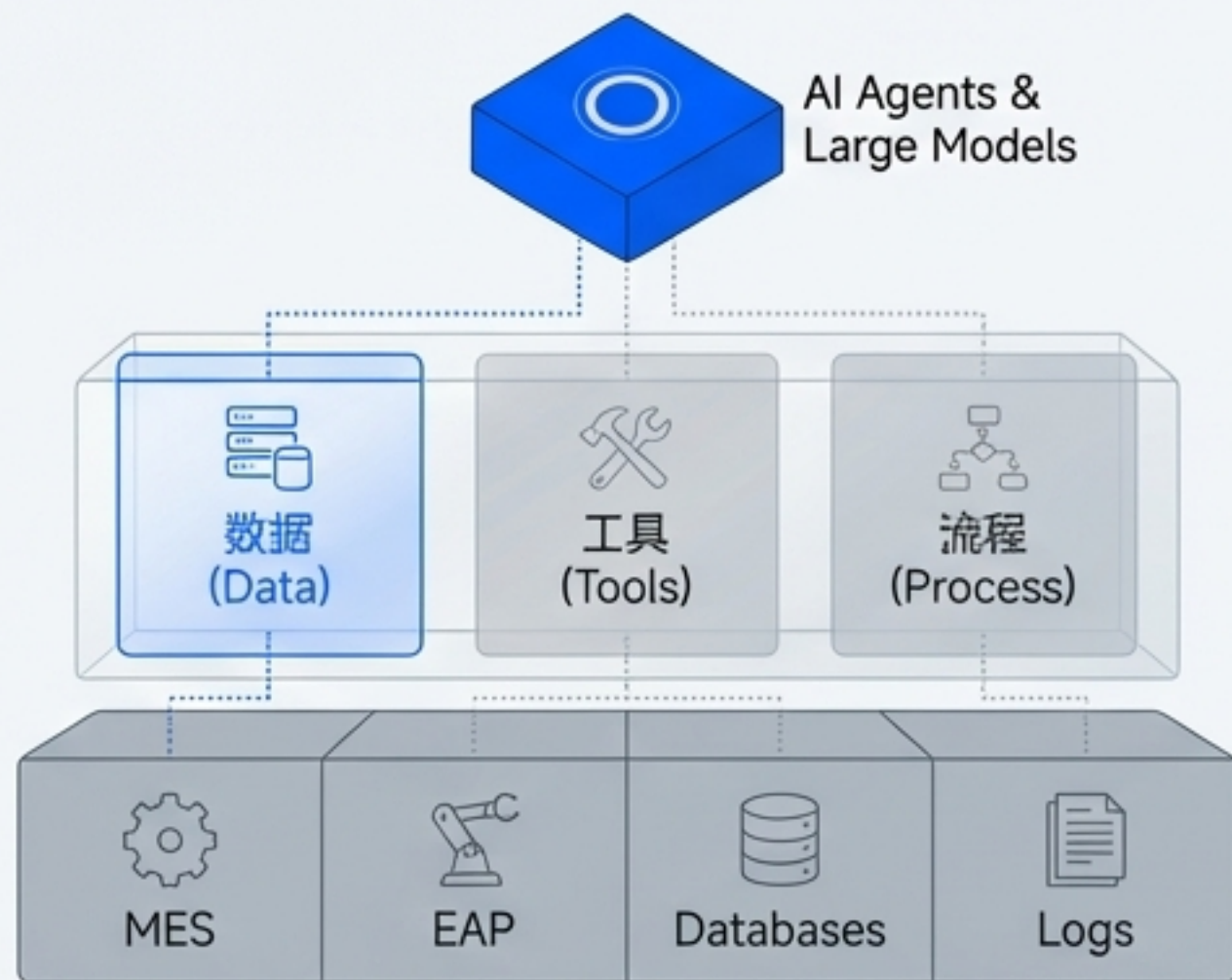


碎片

上下文工程

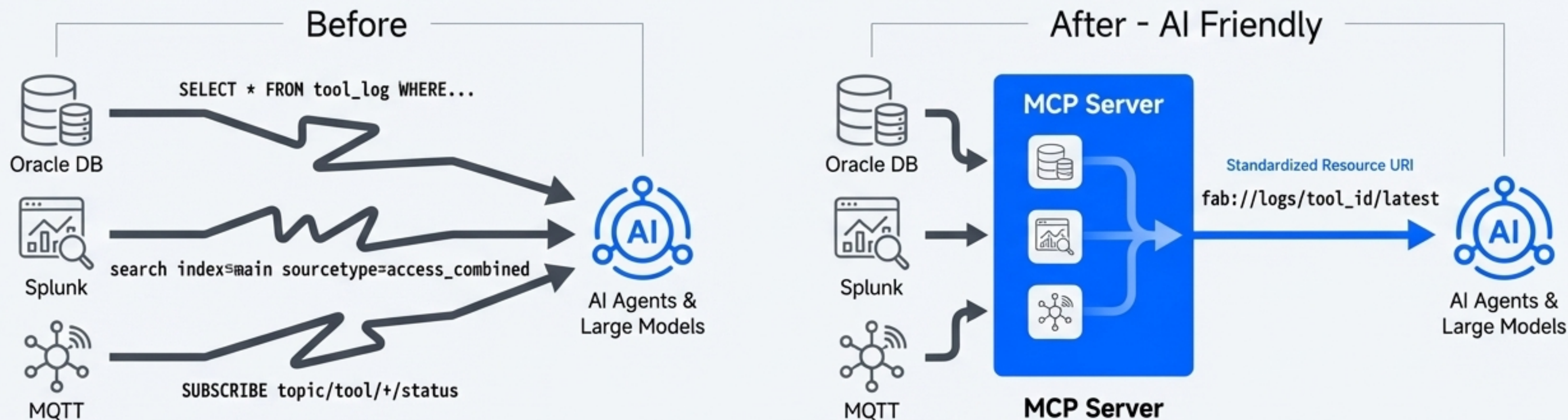


洞察



原则：从“数据库查询”转向“上下文资源订阅”

建立数据对大模型的接口标准化。我们应当在数据源之上建立 **MCP** (Model Context Protocol) Servers, 将 Fab 的数据抽象为 **Resources** (资源)。

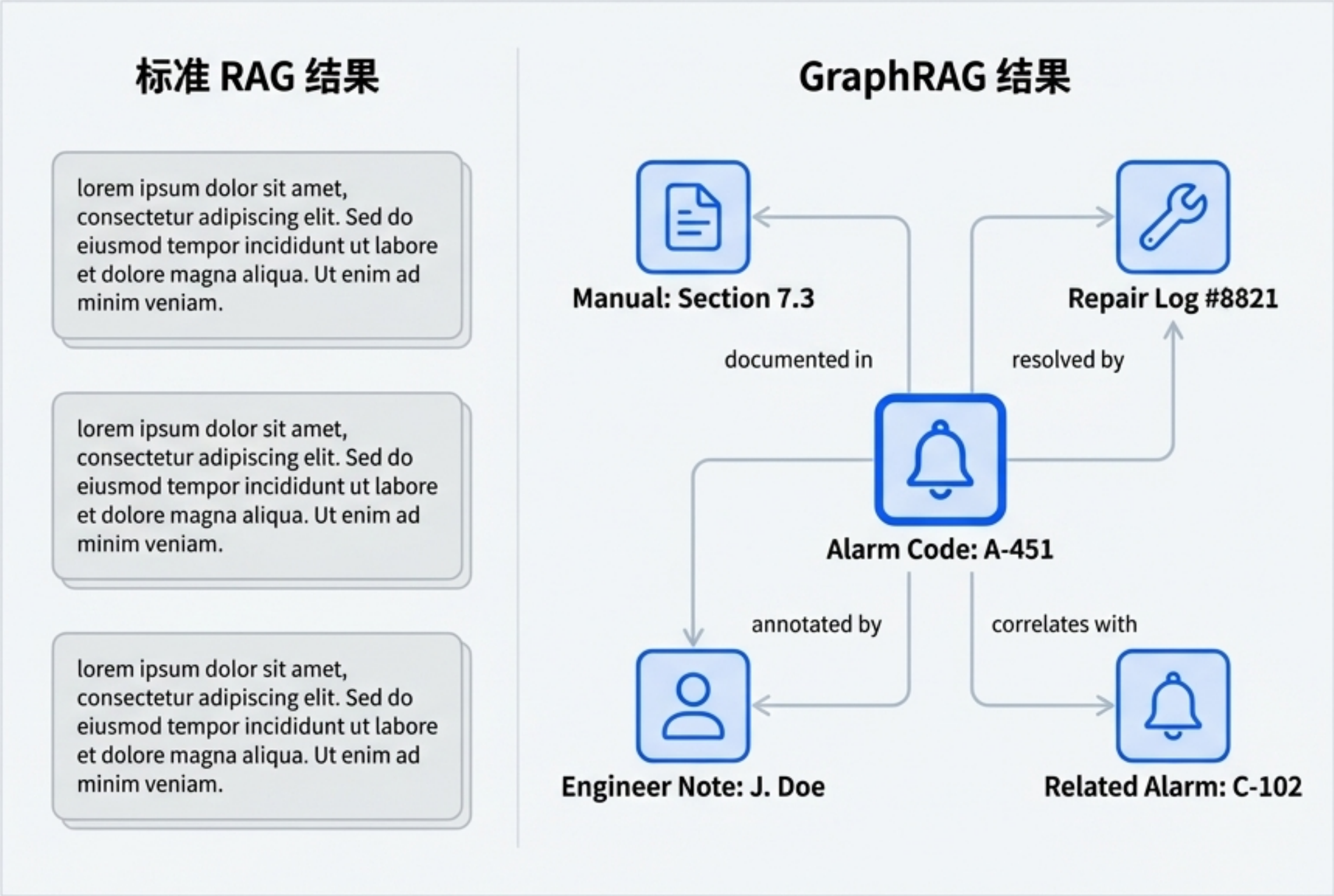


模型不再需要理解底层表结构，只需“订阅”资源。这使得交互更安全、更稳定，并能有效避免模型产生幻觉。

蓝图：引入知识图谱，实现 GraphRAG 增强检索

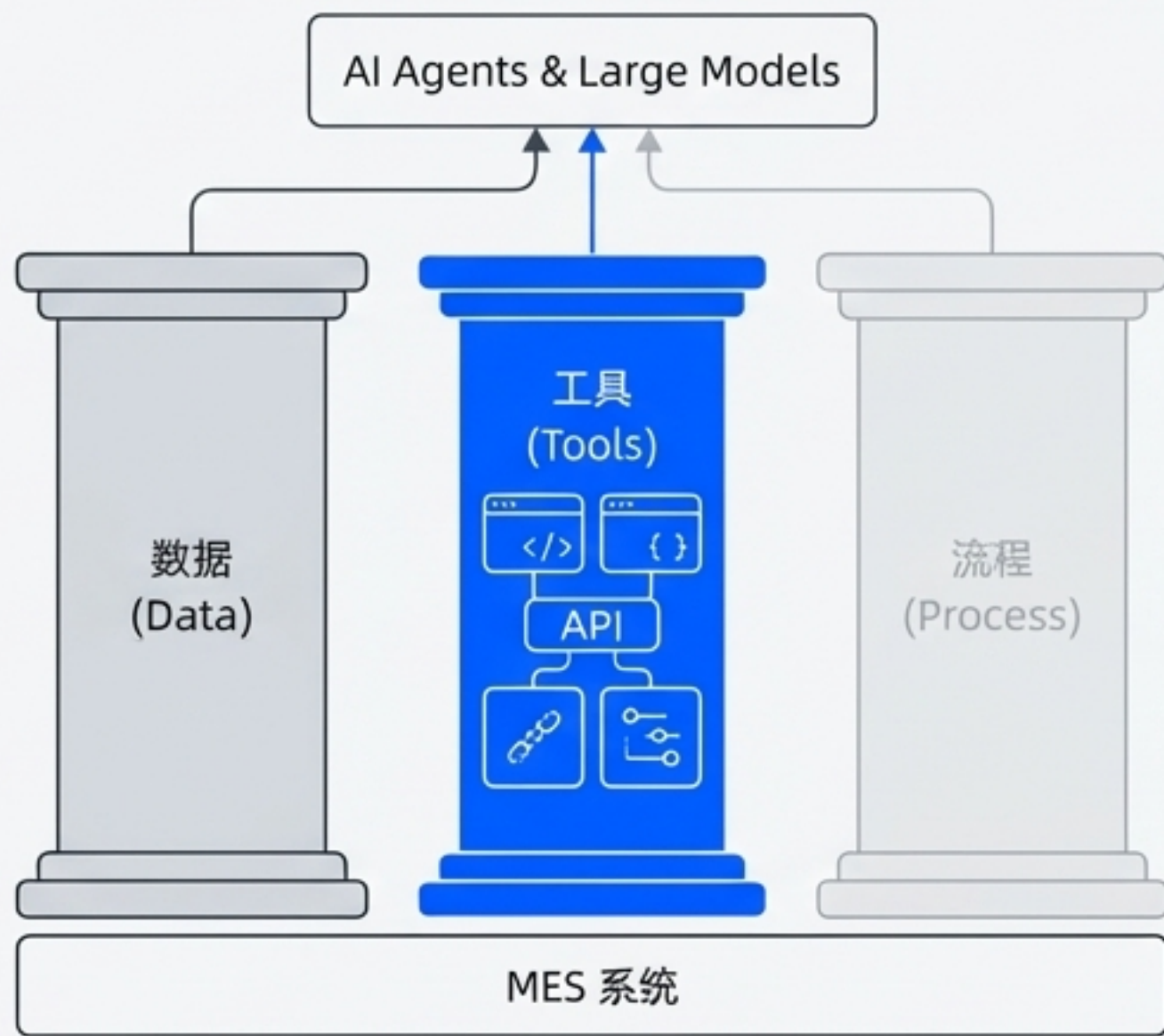
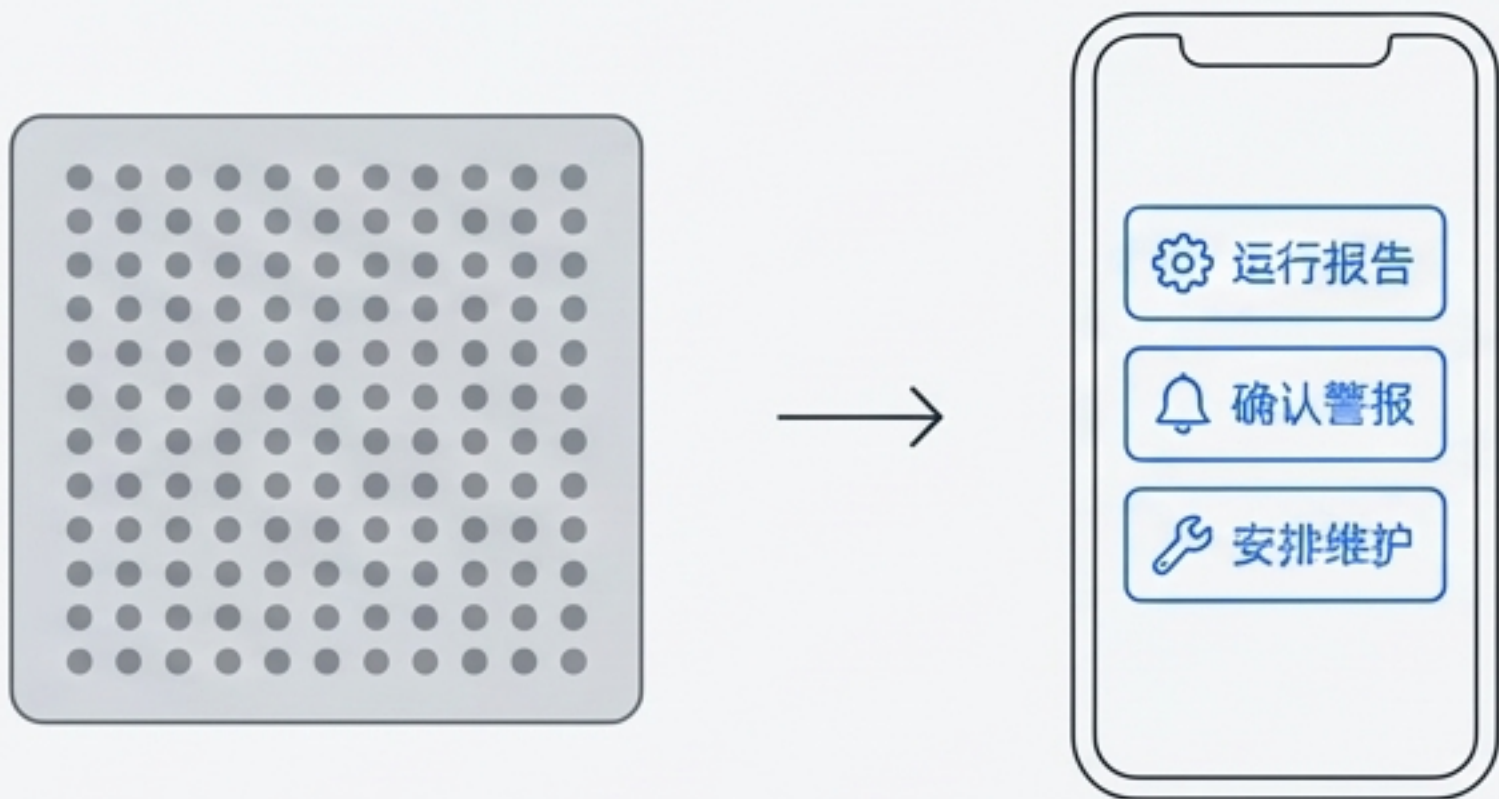
工厂的知识是网状的，而非线性的。一个报警代码关联着手册、维修记录和工程师笔记。单纯的文本检索无法捕捉这些关系。

将非结构化文档转化为（实体，关系，实体）的知识图谱三元组。当 AI 查询时，它获得的不再是破碎的文本段落，而是一张包含因果关系的“故障树”。



鸿沟：数千个复杂 API， 是赋能的工具还是停线的风险？

将 MES 系统中数千个 API 直接暴露给 AI，模型会彻底混淆，甚至可能调用错误指令导致生产中断。这是绝对不能接受的。我们需要建立软件对大模型的接口标准化。



原则：从“复杂 API 调用”转向“原子化技能编排”

Pillar 1: 工具定义的标准化 (The Schema Standard)

无论后端技术多老旧，暴露给 AI 的接口必须符合 **OpenAI JSON Schema** 标准。每个工具必须有清晰的 **description**（不仅是“做什么”，更是“何时用”）和**强类型参数**。

✗ 模糊的接口

```
function: update_params(json)
description: "Updates parameters."
```

Pillar 2: 技能的原子化 (Atomic Skills)

拒绝“一键解决所有问题”的核按钮。利用 **AI SDK** 的设计思想，将复杂业务逻辑拆解为**最小可用单元**。

✓ 清晰的、自解释的技能

```
function: adjust_wafer_lot_priority
description: "Use when an urgent wafer lot needs to be prioritized for processing."
parameters:
  lot_id: string
  priority: enum['high', 'normal']
```


蓝图：分层设计与中间件管控

所有的工具调用，必须经过一个统一的 **Middleware（中间件）** 进行拦截、验证和记录。这才是企业级的 AI Friendly：既开放能力，又严控边界。

L3（Action | 执行层）：

改变系统状态，必须加入人工确认环节。

create_
maintenance_ticket

L2（Analysis | 分析层）：

计算密集型，有资源消耗。

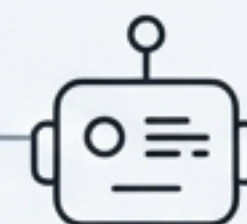
compare_shift_efficiency
diagnose_root_cause

L1（Read-Only | 只读层）：

零风险，可完全开放给 AI。

get_machine_status
read_yield_report

Middleware /
Human-in-the-loop



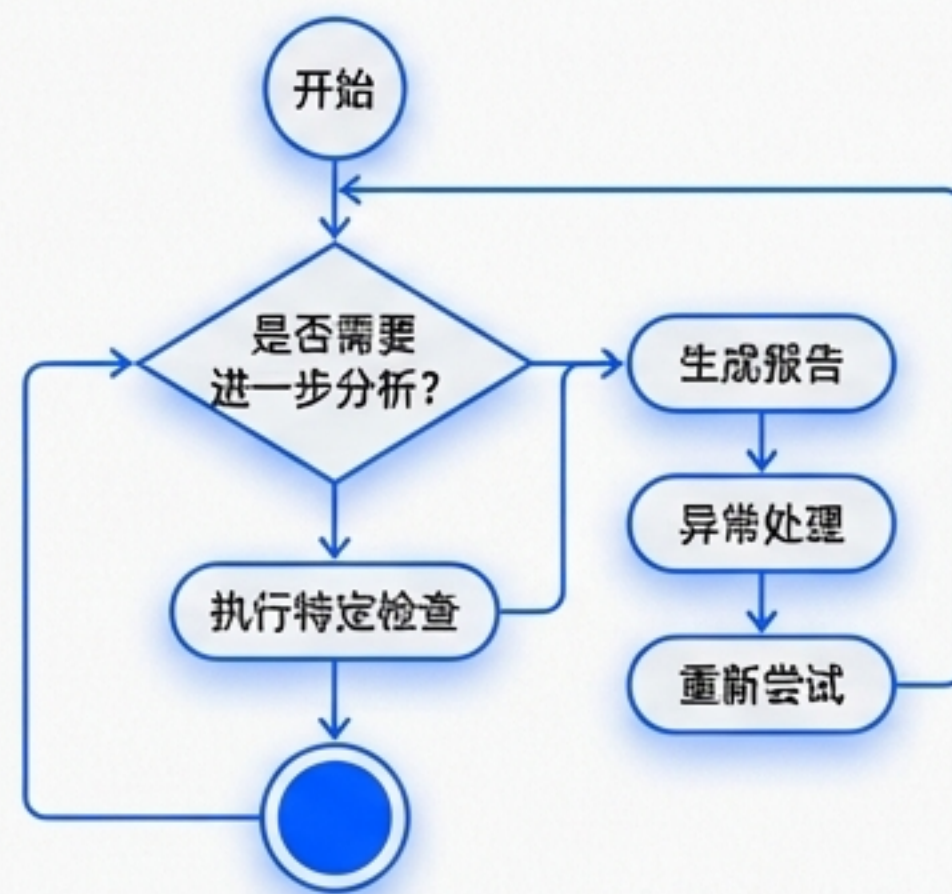
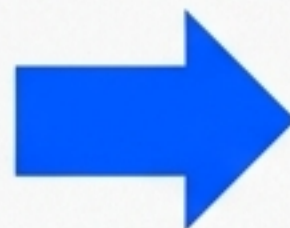
AI Agent

鸿沟：如何让概率性的 AI 执行确定性的 SOP?

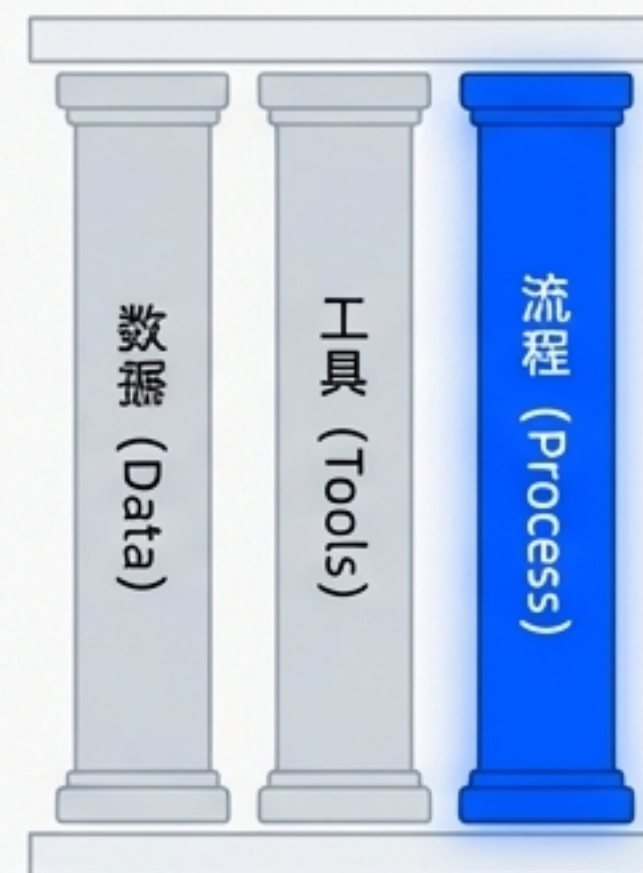
工厂的 SOP 是严格的，环环相扣。AI 的本质是概率模型。我们不能期望仅靠一段自然语言 Prompt，就能让 AI 可靠地完成复杂的良率分析或故障排查流程。我们需要 Prompt 的标准化与流程的代码化。



确定性的 SOP



概率性的 AI 流程



AI Friendly 架构

原则：从“僵化 SOP 文档”转向“概率性流程编排”

引入 **Agent Orchestration** 框架，将 SOP 建模为 **状态机图 (State Graph)**。这使得流程变得可执行、可监控、可适应。

使用 **LangGraph** 等工具，将流程定义为代码。

- 节点 A: 收集数据 (调用 L1 工具)
- 节点 B: AI 分析数据 (调用 L2 工具)
- **关键路由逻辑**: 如果 AI 对结果不确定 (e.g., Confidence < 0.8), 图结构自动路由到“Human-in-the-loop”节点, 请求工程师介入, 而不是强行继续。



蓝图：Prompt 工程化与标准化

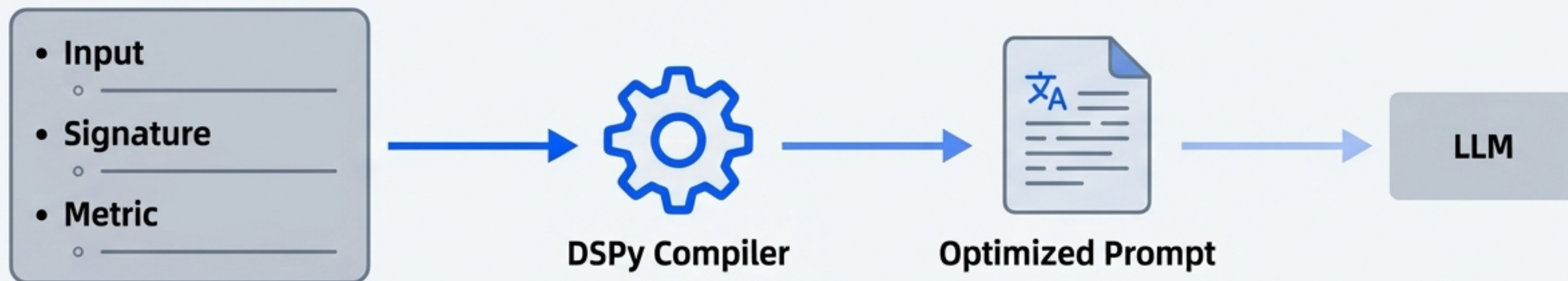
手写 Prompt 的困境

在生产环境中，让每个工程师手写和维护 Prompt 是不可持续的，尤其是在模型更新换代时。

工程化方案（DSPy）

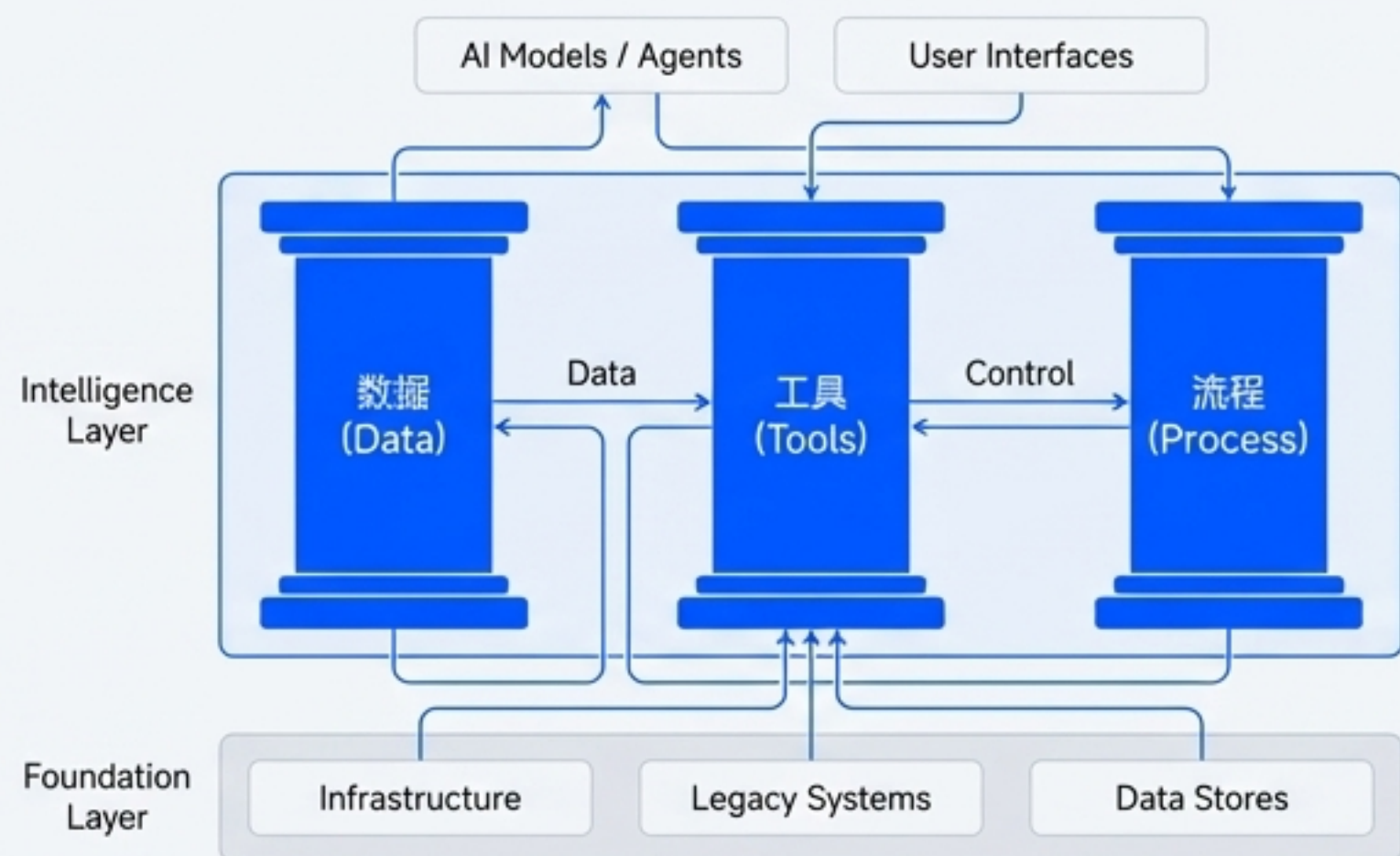
核心理念：我们不再手写冗长的提示词，而是定义**输入（Input）**、**输出签名（Signature）**和**评估指标（Metric）**。

编译过程：让 **Prompt Optimizer** 自动“编译”出针对特定模型最高效的提示词。



当模型从 GPT-4 升级到 Claude 3.5 时，我们无需重写成百上千个 Prompt，只需重新“编译”一次。这才是工业级的可维护性。

架构总结：完整的“Fab 智能适配层”蓝图



数据层 (Data Layer)

使用 **MCP 协议** 让数据“开口说话”，从孤立的数据库变为 AI 可消费的标准化资源。

工具层 (Tools Layer)

使用 **Standard Schema** 和**原子化设计**封装老旧 API，为 AI 提供安全、可控的手脚。

流程层 (Process Layer)

使用 **LangGraph** 和 **DSPy** 将 SOP 代码化，让僵化流程变为可执行、可监控的智能工作流。

展望：AI 就绪的未来工厂

未来的工厂，物理世界依然是轰鸣的机器和忙碌的晶圆传输系统。但在数字世界里，将有成百上千的专用 AI Agents，遵循着我们今天讨论的统一协议，像经验丰富的老工程师一样，7x24 小时守护着工厂的良率与效率。

这不是另起炉灶，而是我们现有数字化资产的价值升华。这就是我们智能化转型的下一步。

